

ADS-509 Assignment 3.1

Bhrami Zadafiya

Word Embeddings

```
In [5]: import os, re, string, random, math, warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import ast

from tqdm import tqdm

# Text preprocessing
import nltk
try:
    nltk.data.find('tokenizers/punkt')
except LookupError:
    nltk.download('punkt')
try:
    nltk.data.find('corpora/stopwords')
except LookupError:
    nltk.download('stopwords')

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Embeddings
from gensim.models import Word2Vec
from sentence_transformers import SentenceTransformer

# ML
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.decomposition import PCA
from sklearn.metrics.pairwise import cosine_similarity

plt.rcParams['figure.figsize'] = (8,5)
plt.rcParams['figure.dpi'] = 120
```

Load Data

Next we will load our dataset from Module 2 and double check that it is formatted correctly.

If you are uncertain about your own dataset, or if you don't pass the check below, feel free to use the dataset provided on Canvas.

```
In [6]: DATA_PATH = 'D:/masters/Applied LLMs for Data Science/module2/hn_comment_features.  
assert os.path.exists(DATA_PATH), f"Dataset not found at {DATA_PATH}. Update the pa
```

```
In [8]: print(df.columns)
```

```
Index(['story_id', 'title', 'comment_id', 'user', 'text_norm',  
      'n_tokens_clean', 'sent_compound'],  
      dtype='str')
```

```
In [10]: print(df["n_tokens_clean"].head())
```

```
0      32  
1      48  
2      66  
3     110  
4      32  
Name: n_tokens_clean, dtype: int64
```

```
In [12]: stop_words = set(stopwords.words('english'))
```

```
def clean_tokens(text):  
    text = str(text).lower()  
    text = re.sub(r'^a-zA-Z\s', '', text)  
    tokens = word_tokenize(text)  
    tokens = [w for w in tokens if w not in stop_words]  
    return tokens  
  
df["tokens_clean"] = df["text_norm"].apply(clean_tokens)  
  
df.head()
```

Out[12]:

	story_id	title	comment_id	user	text_norm	n_tokens_clean	sent_comp
0	48137145	RTX 5090 and M4 MacBook Air: Can It Game?	48137577	frollogaston	I'm guessing the x86 emu is cause Windows game...	32	C
1	48137145	RTX 5090 and M4 MacBook Air: Can It Game?	48137618	moralestapia	Wow, phenomenal project and write-up, thanks f...	48	C
2	48134743	Computer Hobby Movement in Canada	48136386	buescher	I got a VIC-20 when I was about 12? Jim Butte...	66	-C
3	48134743	Computer Hobby Movement in Canada	48137325	bch	I didn't realize he was Canadian - as a child ...	110	C
4	48134743	Computer Hobby Movement in Canada	48136690	reaperducer	for some reason resident debuggers were freque...	32	C

Create a label for classification

The dataset that we scraped from HackerNews doesn't have an obvious variable for use in a classification task (which we will need later). There are many ways that we could create such a label, but here we will use string matching to create some rough labels based on the words used in the article titles.

Q: Give at least two other ideas for how we could label our dataset for classification. Keep data balance in mind (i.e. the resulting dataset should not be completely dominated by one label) in a brief discussion of why that method would/wouldn't be a good choice.

A: Two other possible ways to label the dataset for classification are:

1. Sentiment-based labels: We could label comments or stories as positive, negative, or neutral using the sent_compound sentiment score already available in the dataset. For example, scores above 0.05 could be labeled positive, below -0.05 negative, and the rest

neutral. This would work well because sentiment analysis is a common NLP classification task. However, the dataset may become imbalanced if most Hacker News comments are neutral, so threshold tuning or sampling methods may be needed.

2. Popularity-based labels: We could classify stories into categories such as "high-engagement" and "low-engagement" based on the number of comments or points received. This would allow prediction of which topics attract more attention. A good balance could be achieved by using percentile cutoffs (for example, top 50% vs bottom 50%). This method is useful because engagement is measurable and meaningful, but it may depend on having enough variation in popularity values.

Another possible idea is topic-based labeling using clustering or keyword groups such as cybersecurity, startups, programming, and AI. This could create more diverse classes, although balancing categories may require careful keyword selection.

```
In [15]: # Build a simple label from the story title
def label_from_title(title):
    if not isinstance(title, str):
        return None
    t = title.strip().lower()
    if any(kw in t for kw in ['rust', 'python', 'sql', 'linux', 'windows', 'ios', '']):
        return 'programming'
    if any(kw in t for kw in ['google', 'facebook', 'meta', 'apple', 'amazon', 'microsoft']):
        return 'big-tech'
    if any(kw in t for kw in ['ai', 'torch', 'llm', 'large language model', 'cloud']):
        return 'ai'
    else:
        return 'other'
    return None

df['label'] = df['title'].apply(label_from_title)

print('Total rows for task:', len(df))
print(df['label'].value_counts())

df[['title', 'text_norm', 'label']].head(3)
```

Total rows for task: 5665

label

other	2839
-------	------

ai	1315
----	------

programming	816
-------------	-----

big-tech	695
----------	-----

Name: count, dtype: int64

Out[15]:

	title	text_norm	label
0	RTX 5090 and M4 MacBook Air: Can It Game?	I'm guessing the x86 emu is cause Windows game...	other
1	RTX 5090 and M4 MacBook Air: Can It Game?	Wow, phenomenal project and write-up, thanks f...	other
2	Computer Hobby Movement in Canada	I got a VIC-20 when I was about 12? Jim Butte...	other

Train Static Word Embedding Model

We will train a Word2Vec model to produce static word embeddings for our normalized text, which we will use later for data exploration and classification.

TODO:

Use the gensim Word2Vec class to train a static embedding model on our tokenized comment text. Check out the [documentation](#) to assign the following settings (Hint: You might need to go into the class source code to find argument descriptions):

- Embedding size 100
- Sequence window size 5
- Limit to tokens that appear at least 3 times
- Skip-gram algorithm (rather than CBOW)
- Run training for 10 epochs
- If you have multiple CPUs available, set the number of workers to improve training speed

Q: Why do we call a model like Word2Vec a *static* word embedding model?

A: Word2Vec is called a static word embedding model because each word is assigned a single fixed vector representation regardless of the context in which the word appears. For example, the word "bank" will always have the same embedding whether it refers to a financial institution or the side of a river. The embeddings do not change dynamically based on surrounding words.

Q: What is the difference between the CBOW and skip-gram algorithms?

A: CBOW (Continuous Bag of Words) predicts a target word using the surrounding context words. It is generally faster and performs well on large datasets.

Skip-gram works in the opposite direction by predicting surrounding context words from a target word. It is usually slower but performs better for smaller datasets and rare words because it learns more detailed word relationships.

```
In [16]: # Create tokenized sentences from normalized text
stop_words = set(stopwords.words('english'))

def clean_tokens(text):
    text = str(text).lower()
    text = re.sub(r'^a-zA-Z\s', '', text)
    tokens = word_tokenize(text)
    tokens = [w for w in tokens if w not in stop_words]
    return tokens

# Generate token lists
sentences = df["text_norm"].apply(clean_tokens).tolist()

# Train Word2Vec model
w2v = Word2Vec(
    sentences=sentences,
    vector_size=100, # embedding size
    window=5, # context window size
    min_count=3, # minimum token frequency
    sg=1, # skip-gram (1 = skip-gram, 0 = CBOW)
    epochs=10,
    workers=os.cpu_count()
)

# Access Learned vectors
w2v_vecs = w2v.wv

print("Vocabulary size:", len(w2v_vecs))
print("Vector size:", w2v_vecs.vector_size)
```

Vocabulary size: 7034

Vector size: 100

The word embeddings that we just created can now be used to perform a semantic comparison of individual words/tokens in our vocabulary.

TODO:

- Choose a few words from our corpus vocabulary and print the 5 nearest neighbors using the `Word2Vec.most_similar()` function

Q: Are the nearest neighbors semantically similar to your chosen words? Think about how the Word2Vec model works and provide a 2-3 sentence description of why this comparison does/doesn't work for our dataset.

A: Yes, many of the nearest neighbors are semantically or contextually related to the chosen words. For example, words such as "tensorflow," "IIm," or "neural" may appear close to "ai" because they frequently occur in similar contexts within Hacker News discussions.

This works because Word2Vec learns word relationships based on surrounding words in sentences. Words that appear in similar contexts develop similar vector representations.

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

The model compares embeddings using cosine similarity, so words used in related discussions become close together in vector space. However, results may sometimes be noisy because the dataset is relatively small and contains informal online discussions with abbreviations and inconsistent language.

```
In [17]: # Choose a few words from the vocabulary
probe_terms = ['python', 'ai', 'google', 'linux', 'model']

# Print 5 nearest neighbors for each word
for term in probe_terms:
    if term in w2v_vecs:
        print(f"\nNearest neighbors for '{term}':")

        neighbors = w2v_vecs.most_similar(term, topn=5)

        for word, score in neighbors:
            print(f"{word:15s} similarity={score:.3f}")
    else:
        print(f"\n'{term}' not found in vocabulary")
```

```
Nearest neighbors for 'python':
skills          similarity=0.891
libraries       similarity=0.886
vim             similarity=0.880
editor          similarity=0.879
workflow        similarity=0.877
```

```
Nearest neighbors for 'ai':
race            similarity=0.720
llms            similarity=0.717
bubble          similarity=0.698
openai          similarity=0.686
infrastructure  similarity=0.685
```

```
Nearest neighbors for 'google':
ide             similarity=0.820
eclipse         similarity=0.776
intellij        similarity=0.774
developer       similarity=0.758
antigravity     similarity=0.756
```

```
Nearest neighbors for 'linux':
steam          similarity=0.761
anticheat      similarity=0.751
windows        similarity=0.747
deck           similarity=0.744
cards          similarity=0.743
```

```
Nearest neighbors for 'model':
deepseek       similarity=0.846
qwen           similarity=0.823
tiny           similarity=0.818
train          similarity=0.798
outputs        similarity=0.795
```

Build Document Embeddings

One way to represent a document (comment) in our dataset is to aggregate all of the individual word embeddings by computing an average embedding or document vector. This vector can then be used to compare entire documents instead of individual words.

TODO:

- Average the word/token embeddings for each comment in our dataset to produce a single document embedding vector
- Compute the cosine similarity between the first comment's document embedding and all of the rest
- Print out the two most and least similar comments

Q: Describe your results. Does the cosine similarity of document/comment embeddings do a good job of identifying similar content?

A:

The cosine similarity results show that the document embeddings are reasonably effective at identifying related comments. The most similar comments discuss similar topics related to Windows and technology, which suggests that averaging the word embeddings captures general semantic meaning from the comments.

The least similar comments are mostly standalone URLs with very little contextual text, resulting in similarity scores close to 0. This happens because links contain few meaningful tokens, making their document embeddings very different from normal text comments.

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

Overall, cosine similarity with averaged Word2Vec embeddings works fairly well for identifying broad topical similarity, but it may miss deeper contextual meaning because averaging removes word order and sentence structure information.

```
In [18]: df["tokens_clean"] = df["text_norm"].apply(clean_tokens)

# Average word embeddings for each document/comment
def docvec_average(tokens, wv):
    vecs = [wv[t] for t in tokens if t in wv]

    if not vecs:
        return np.zeros(wv.vector_size, dtype=np.float32)

    return np.mean(vecs, axis=0)

# Create document embeddings
df["w2v_embedding"] = list(
    np.vstack([docvec_average(toks, w2v_vecs) for toks in df['tokens_clean']])
)

# Compute cosine similarity between first comment and all others
first_vec = df["w2v_embedding"].iloc[0].reshape(1, -1)

all_vecs = np.vstack(df["w2v_embedding"].values)

similarities = cosine_similarity(first_vec, all_vecs)[0]

df["similarity"] = similarities

# Most similar comments
most_similar = df.sort_values("similarity", ascending=False).head(3)

# Least similar comments
least_similar = df.sort_values("similarity", ascending=True).head(2)

print("\nMost Similar Comments:")
print(most_similar[["text_norm", "similarity"]])
```

```
print("\nLeast Similar Comments:")
print(least_similar[["text_norm", "similarity"]])
```

Most Similar Comments:

		text_norm	similarity
0	I'm guessing the x86 emu is cause Windows game...		1.000000
5550	Windows 7 was the best Windows and one of the ...		0.977899
898	Given the current momentum, it feels like (to ...		0.975319

Least Similar Comments:

		text_norm	similarity
5639	https://archive.org/details/tvtcb_doc		0.0
5632	https://gitgud.io/aeroshell/atp/aerothemeplasma		0.0

In [19]: *# Compute cosine similarity between the first comment and all others*

```
def docvec_similarity(target, docvecs):
    sims = cosine_similarity(
        target.reshape(1, -1),
        np.vstack(docvecs)
    )[0]

    return sims

# Create similarity column
df["w2v_sim"] = [None] + list(
    docvec_similarity(
        df["w2v_embedding"][0],
        df["w2v_embedding"][1:]
    )
)

df[["text_norm", "w2v_sim"]].head()
```

Out[19]:

		text_norm	w2v_sim
0	I'm guessing the x86 emu is cause Windows game...		NaN
1	Wow, phenomenal project and write-up, thanks f...		0.831283
2	I got a VIC-20 when I was about 12? Jim Butte...		0.909298
3	I didn't realize he was Canadian - as a child ...		0.899404
4	for some reason resident debuggers were freque...		0.925337

In [21]: *# Print out the target comment and the two comments*
with the highest and lowest cosine similarities

```
print("Target Comment:")
print(df["text_norm"][0])

print("\nLowest Similarities:")
print(df.sort_values("w2v_sim")[["text_norm", "w2v_sim"]].head(2))
```

```
print("\nHighest Similarities:")
print(df.sort_values("w2v_sim", ascending=False)[["text_norm", "w2v_sim"]].head(2))
```

Target Comment:

I'm guessing the x86 emu is cause Windows games are rarely built for ARM, right? Was kinda curious how an ARM VM would fare. Anyway awesome article.

Lowest Similarities:

	text_norm	w2v_sim
2373	How about \$299?	0.0
1621	No.	0.0

Highest Similarities:

	text_norm	w2v_sim
5550	Windows 7 was the best Windows and one of the ...	0.977899
898	Given the current momentum, it feels like (to ...	0.975319

Build Contextual Embeddings with Sentence-Transformers

Another way to create a document-level embedding is to use a more sophisticated, pre-trained embedding model, like a Sentence Transformer (based on the BERT family architecture). These models use an *attention block* to create context-specific embeddings for a string of text, rather than simply averaging the static word embeddings as we did above.

TODO:

- Use the gensim SentenceTransformer class to load the pre-trained 'sentence-transformers/all-MiniLM-L6-v2' model
- Use this model to create embeddings for our comment text (HINT: You don't need to normalize or tokenize text before feeding it into a transformer model)

```
In [22]: # Use a pre-trained SentenceTransformer model
model_name = 'sentence-transformers/all-MiniLM-L6-v2'

st_model = SentenceTransformer(model_name)

# Create contextual/document embeddings
df["bert_embedding"] = list(
    st_model.encode(
        df["text_norm"].tolist(),
        show_progress_bar=True
    )
)

# Check one embedding
print("Embedding shape:", df["bert_embedding"][0].shape)
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Loading weights: 0%| | 0/103 [00:00<?, ?it/s]
 Batches: 0%| | 0/178 [00:00<?, ?it/s]

Embedding shape: (384,)

Now we will compute the same cosine similarities as we did with the Word2Vec embeddings above.

Q: How do the transformer-based embedding similarities compare to what you found above? Which method would you choose and why?

A: Transformer-based embeddings generally produce more accurate semantic similarities than averaged Word2Vec embeddings. The Sentence Transformer model considers the full sentence context and word relationships, so comments with similar meanings are often matched better even when they do not share the same keywords.

I would choose transformer-based embeddings because they capture contextual meaning and sentence structure more effectively. Averaged Word2Vec embeddings are simpler and faster, but they lose important contextual information since each word has a fixed static representation.

```
In [23]: # Create new column with cosine similarities
df["bert_sim"] = [None] + list(
    docvec_similarity(
        df["bert_embedding"][0],
        df["bert_embedding"][1:]
    )
)

# Preview results
df[["text_norm", "bert_sim"]].head()
```

```
Out[23]:
```

	text_norm	bert_sim
0	I'm guessing the x86 emu is cause Windows game...	NaN
1	Wow, phenomenal project and write-up, thanks f...	0.154301
2	I got a VIC-20 when I was about 12? Jim Butte...	0.237935
3	I didn't realize he was Canadian - as a child ...	0.174489
4	for some reason resident debuggers were freque...	0.297512

```
In [24]: print("Target Comment:")
print(df["text_norm"][0])

print("\nLowest Similarities:")
print(df.sort_values("bert_sim")[["text_norm", "bert_sim"]].head(2))

print("\nHighest Similarities:")
print(df.sort_values("bert_sim", ascending=False)[["text_norm", "bert_sim"]].head(2))
```


Target Comment:

I'm guessing the x86 emu is cause Windows games are rarely built for ARM, right? Was kinda curious how an ARM VM would fare. Anyway awesome article.

Lowest Similarities:

		text_norm	bert_sim
796	You wrap the DNS request in a different layer ...	-0.179949	
784	no, you are actually telling the relay where t...	-0.161895	

Highest Similarities:

		text_norm	bert_sim
5657	That was the very same reaction of many early ...	0.499916	
2226	I did read between the lines here: > After tha...	0.498889	

Embedding-based classification

Finally, we will explore how well the document embeddings perform on the classification task of predicting our comment category labels. We will use the two types of embeddings for input to two different logistic regression models, and then compare the performance.

TODO:

- Use the sklearn LogisticRegression class to predict our comment labels using both embedding methods

Q: How do the two embedding methods perform? Use the results from the model metrics and the confusion matrices to discuss and compare.

A: The Sentence-BERT embeddings generally achieve higher accuracy, precision, recall, and F1-scores compared to the averaged Word2Vec embeddings. The confusion matrix also typically shows fewer classification errors between related categories such as “ai” and “programming.”

SentenceTransformers performs better because it creates contextual embeddings that capture sentence meaning and relationships between words. In contrast, Word2Vec uses static embeddings and averaging them loses word order and contextual information. However, Word2Vec is still faster and computationally cheaper, making it useful for simpler NLP applications.

```
In [26]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

# Train/test split
Xw_train, Xw_test, y_train, y_test = train_test_split(
    df['w2v_embedding'],
    df['label'],
    test_size=0.2,
    random_state=42,
```

```

    stratify=df['label']
)

Xb_train, Xb_test, _, _ = train_test_split(
    df['bert_embedding'],
    df['label'],
    test_size=0.2,
    random_state=42,
    stratify=df['label']
)

# Standardize Word2Vec embeddings
sc_w2v = StandardScaler(with_mean=False)

Xw_train_s = sc_w2v.fit_transform(
    np.stack(Xw_train.to_numpy())
)

Xw_test_s = sc_w2v.transform(
    np.stack(Xw_test.to_numpy())
)

# Standardize BERT embeddings
sc_bert = StandardScaler(with_mean=False)

Xb_train_s = sc_bert.fit_transform(
    np.stack(Xb_train.to_numpy())
)

Xb_test_s = sc_bert.transform(
    np.stack(Xb_test.to_numpy())
)

```

```

In [27]: # Word2Vec Logistic Regression
clf_w2v = LogisticRegression(max_iter=2000)

clf_w2v.fit(Xw_train_s, y_train)

pred_w2v = clf_w2v.predict(Xw_test_s)

# Sentence-BERT Logistic Regression
clf_bert = LogisticRegression(max_iter=2000)

clf_bert.fit(Xb_train_s, y_train)

pred_bert = clf_bert.predict(Xb_test_s)

# Results
print("\n=== Word2Vec Avg → Logistic Regression ===\n")
print(classification_report(y_test, pred_w2v, digits=3))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred_w2v))

print("\n=== Sentence-BERT → Logistic Regression ===\n")
print(classification_report(y_test, pred_bert, digits=3))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred_bert))

```

=== Word2Vec Avg → Logistic Regression ===

	precision	recall	f1-score	support
ai	0.692	0.487	0.571	263
big-tech	0.553	0.302	0.391	139
other	0.638	0.847	0.728	568
programming	0.695	0.503	0.584	163
accuracy			0.647	1133
macro avg	0.644	0.535	0.568	1133
weighted avg	0.648	0.647	0.629	1133

Confusion Matrix:

```
[[128  6 126  3]
 [ 10 42  78  9]
 [ 38 25 481 24]
 [  9  3  69 82]]
```

=== Sentence-BERT → Logistic Regression ===

	precision	recall	f1-score	support
ai	0.660	0.605	0.631	263
big-tech	0.562	0.525	0.543	139
other	0.739	0.801	0.769	568
programming	0.760	0.681	0.718	163
accuracy			0.704	1133
macro avg	0.680	0.653	0.665	1133
weighted avg	0.702	0.704	0.702	1133

Confusion Matrix:

```
[[159 15 82  7]
 [ 12 73 50  4]
 [ 57 32 455 24]
 [ 13 10 29 111]]
```

In []: